Inheritance

The Legacy of Code

Structural Hierarchy & Component Reuse

PARENT-CHILD • REUSABILITY • MULTI-LAYERED

1. Passing the Torch

Inheritance is a mechanism in C++ where a new class (the **Child**) acquires the properties and behaviors of an existing class (the **Parent**).

Standard Terminology:

- **Base Class (Parent):** The class being inherited from.
- **Derived Class (Child):** The class that inherits the features.

Why Inherit?

- **Reusability:** Don't rewrite the same code for every class.
- **Organization:** Create a logical "Is-A" hierarchy (A Tesla is-a Car).
- **Extension:** Add new features to old code without modifying the original.

2. Implementation Syntax

In C++, inheritance is declared using the colon (`:`) symbol followed by the access level and the parent class name.

```
class Parent {
    // Shared logic here
};

class Child : public Parent {
    // Child-specific logic here
};
```

> **The 'Public' Keyword:** Using `public` inheritance ensures that all public members of the parent stay public in the child.

3. Structural Variations

C++ is unique among languages because it supports multiple complex inheritance strategies:

Type	Structure
Single	A → B
Multiple	A + B → C
Multilevel	A → B → C
Hierarchical	A → B, A → C
Hybrid	Combination (e.g., A → B, C + B → D)

> **Note:** Multiple inheritance (two parents) is powerful but can be tricky due to potential naming conflicts.

TYPE: SINGLE

4. The Standard Link

```
class Course {
public:
    string title = "Generic Course";
    void start() { cout << "Started"; }
};

class CodingHive : public Course {
public:
    int id = 101;
};

int main() {
    CodingHive ch;
    cout << ch.title; // Inherited
    ch.start();       // Inherited
}
```

TYPE: MULTIPLE

5. Multiple Parents

C++ allows a child to have two or more parents. This is useful for combining different skill sets into one class.

```
class Academic { public: float grade; };
class Technical { public: string skill; };

class Student : public Academic, public Technical {
public:
    void show() {
        cout << skill << " / " << grade;
    }
};
```

Pro-Tip: Use Multiple Inheritance sparingly to avoid the "Diamond Problem" (ambiguity in data paths).

TYPE: MULTILEVEL

6. The Chain of Command

Multilevel inheritance represents a vertical lineage—the child of a child.

```
class Animal { public: void breath() {} };
class Mammal : public Animal { public: void warmBlood() {} };
class Dog : public Mammal { public: void bark() {} };

int main() {
    Dog myDog;
    myDog.breath();    // From Grandparent
    myDog.warmBlood(); // From Parent
    myDog.bark();      // Own
}
```

7. Inheritance Access Modes

How you inherit changes the visibility of the parent's logic within the child.

Member in Base	Public Inherit.	Protected Inherit.	Private Inherit.
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Hidden	Hidden	Hidden

8. The 'Protected' Specifier

The `protected` keyword was designed specifically for inheritance. It acts as a middle ground between public and private.

```
class Base {
protected:
    int id = 500; // Locked from main()
};

class Child : public Base {
public:
    void useId() {
        id = 10; // ✅ Allowed: Child can see it
    }
};
```

> Think of **protected** as a "Family Secret"—it stays within the family tree but the general public can't see it.

9. Summary & Best Practices

- **Reuse:** Inherit to avoid repeating boilerplate code.
- **Public Mode:** Use `: public Base` for 90% of use cases.
- **Private Vars:** Always keep variables private in the parent and use protected or public methods to share them.
- **Liskov Rule:** A child should always be able to substitute for its parent.

Skill Challenge:

Create a class `Vehicle` with `speed`. Create a class `Engine` with `horsepower`. Create a child `Car` that inherits from **BOTH**. Test if you can access members from both parents using a single `Car` object.

HIERARCHY VALIDATED: INHERITANCE COMPLETE